RI'AYA

Clinic operations platform

Technical documentation for the engineering team

Version 1.0 · 06 November 2026
Repository: github.com/mrdahdah/riaya (private)
Live demo: https://riaya-xi.vercel.app
Author: Mohamed Reda

Contents

1. Overview 3

2. Architecture 4

3. Technology stack 5

4. Data model 6

5. Backend modules 7

6. API reference (summary) 8

7. Core algorithms 9

8. Front end 10

9. Security and privacy 11

10. Testing and quality 12

11. Deployment and operations 13

12. Known limitations and roadmap 14

1. Overview

RI'AYA is a multilingual (English, Arabic with full right-to-left layout, French), multi-tenant web platform that runs the operational day of a clinic: booking, doctors' availability, patient records, the waiting-room queue, schedule optimisation and reporting. It implements the MediFlow student brief (Smart Clinic & Appointment Optimization System) and the team's Ri'aya feature document, starting from the existing Ri'aya repository (authentication, tenancy, design system and landing page).

Scope delivered

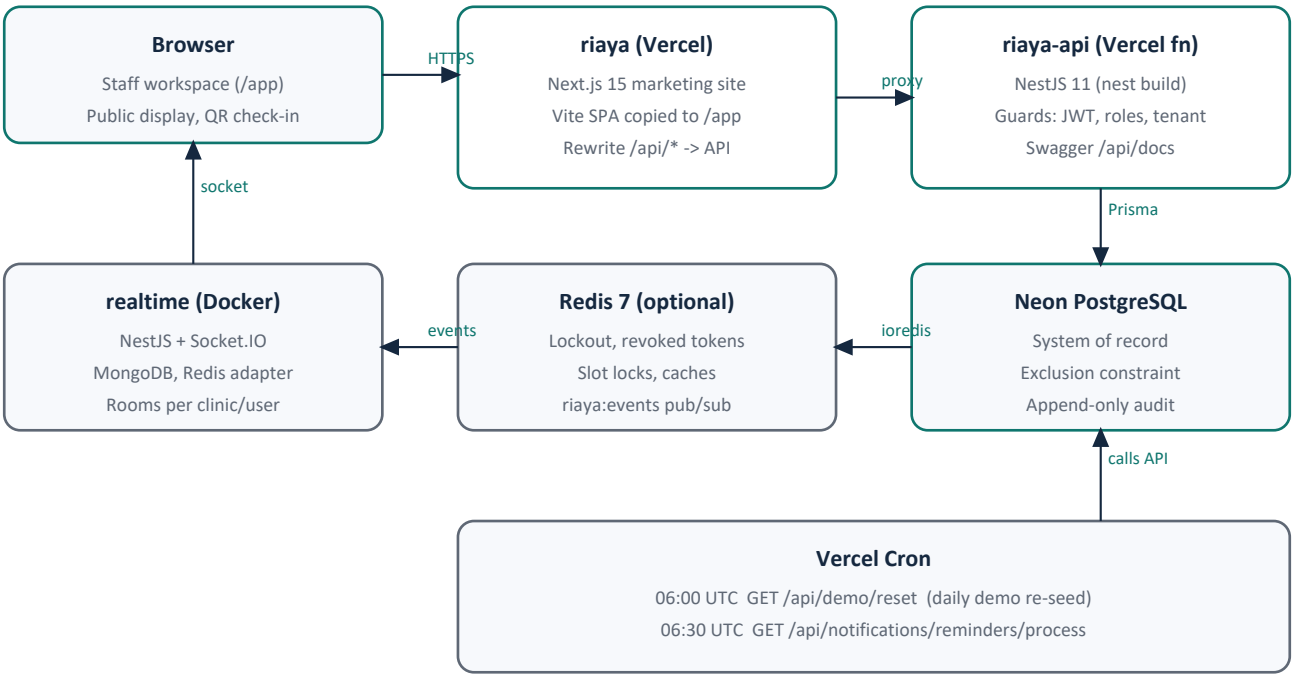
- **Booking:** five-step wizard, slot generator, slot locks (5 min TTL), conflict-safe creation with alternatives, state machine, cancellation policy, bulk actions, reminders with a mock SMS/e-mail provider.
- **Doctors and staff:** cards and table views, utilisation, weekly hours with breaks, leave, conflict list before saving new hours, deactivation that flags future bookings.
- **Patients:** directory with search, duplicate detection, archive, profile tabs, field-level permission on clinical notes.
- **Queue:** desk and QR self check-in, triage priority, race-safe call-next, public waiting-room display without personal data.
- **Smart scheduling:** metrics service, deterministic optimiser with three objectives, preview, atomic apply with stale check, undo within 15 minutes, drag and drop.
- **Analytics:** dashboard, reports with period comparison, clinic or organisation scope, CSV (UTF-8 BOM) and PDF export.
- **Platform:** notifications centre, settings (8 sections), append-only audit log, staff chat, EN/AR/FR across UI and API errors.

Out of scope (per brief)

Real medical records, billing and insurance, real SMS/WhatsApp delivery (mock provider behind an interface), machine-learning models (the optimiser is a deterministic heuristic). All data is fictional.

2. Architecture

A modular monolith (one NestJS API with strictly separated modules) plus a separate realtime service, a React single-page workspace and a Next.js marketing site, in one pnpm monorepo. The hosted demo runs both web and API on Vercel; locally, Docker Compose provides the full brief stack.



Request path

Browser → `/api/*` on the marketing origin → Vercel rewrite to the API project → Nest middleware (helmet, cookie parser, body limits, request id) → JWT guard → roles guard → tenant context (clinic and organisation from the verified token only) → controller → service → Prisma → PostgreSQL. Successful writes emit domain events after commit; consumers update notifications, reminders, caches and the queue, and publish to Redis for the realtime service when it is configured.

Why one origin

The refresh token is an HttpOnly, SameSite=Lax cookie. Serving the workspace at `/app` and proxying `/api` from the same host keeps it first-party, so no third-party-cookie rules apply and no CORS pre-flight is needed in the browser.

Repository layout

Path	Contents
server/	NestJS API: src/<domain>/ (controller, service, DTOs, pure logic + tests), prisma/schema.prisma and migrations, seeds, Vercel function shim api/index.js
backoffice/	React 19 + Vite workspace: src/features/<feature>/ (api.ts, components, pure logic, tests), src/common primitives, src/i18n catalogues
web/	Next.js 15 marketing site with next-intl; next.config.ts rewrites /api and serves /app
realtime/	NestJS Socket.IO relay with JWT handshake, MongoDB, Redis adapter
packages/shared/	Request/response contracts, capability matrix, localised error catalogue (TS source + CJS build)
docs/	API contract, ADRs, threat model, scheduling metrics, this document

3. Technology stack

Layer	Technology	Notes
Front end	React 19, TypeScript 5.8, Vite 6, React Router 7	SPA under /app; route-level code splitting
UI and state	Tailwind CSS 4 tokens, Redux Toolkit + RTK Query, lucide-react, Recharts 3, @dnd-kit/core	One token set; RTK Query over axios for silent refresh
Forms	React Hook Form + Zod	Field-level messages in the user language
i18n	i18next + react-i18next; next-intl on the landing	Per-feature catalogues, parity check in the build
API	NestJS 11, class-validator, @nestjs/swagger, nestjs-pino	Global validation (whitelist), exception filter {code, message, details}
ORM / DB	Prisma 6 on PostgreSQL 16+ (Neon in production)	Exclusion constraint (btree_gist), audit trigger
Cache / locks	Redis 7 via ioredis (optional)	Memory fallback; security state fails closed unless opted in
Realtime	Socket.IO, MongoDB 7, Redis adapter	Docker locally; hosted demo polls
Auth	JWT access 15 min + rotating refresh 7 days, argon2	Lockout 5 failures / 10 min → 15 min
Testing	Jest + Supertest, Vitest + Testing Library, Playwright MCP for journeys	Server 267 tests, backoffice 184, realtime 46
DevOps	pnpm 10 workspaces, Docker Compose, GitHub Actions, ESLint, Prettier, Husky, commitlint, gitleaks	Vercel (two projects), Neon

Running locally

```
pnpm install
pnpm run setup                      # .env files from examples
docker compose up -d                # PostgreSQL, Redis, MongoDB
pnpm --filter server exec prisma migrate deploy
pnpm --filter server seed           # fictional dataset, dates relative to today
pnpm dev                            # web :3000, API :4000 (/api/docs), backoffice :5174
```

4. Data model

PostgreSQL is the system of record. Every clinical table carries `clinicId`; organisations own clinics; users hold one membership (role) per clinic. Migrations live in `server/prisma/migrations` as reviewed SQL.

Model	Purpose	Key constraints / indexes
User, UserMembership, Organization, Clinic, Invitation	Global identity, per-clinic roles, tenancy	membership unique (userId, clinicId); invitation token hashed
RefreshToken, PasswordResetToken	Sessions and reset	token hash unique; family id for reuse detection
Patient	Person receiving care	unique (clinicId, code P-001); indexes on lastName, phone, status
Doctor, Room	Clinicians and bookable rooms	doctor (clinicId, isActive); room unique (clinicId, name)
WorkingHour, DoctorLeave	Weekly WORK/BREAK intervals (minutes after local midnight), leave periods	index (doctorId, weekday)
Appointment	Booked visit with status, type, room, lock flag	unique (clinicId, code AP-1001); EXCLUDE overlap per doctor for live statuses; indexes (doctorId, startsAt), (patientId), (clinicId, startsAt)
QueueEntry	Check-in, priority, call/start/complete times	unique appointmentId; index (clinicId, status)
ClinicSettings	Policies, durations, opening hours, departments, appearance	one row per clinic, created lazily
Notification	In-app feed with i18n keys and params	dedupeKey unique (idempotent); readBy array
ReminderDelivery	Per channel and offset delivery state	unique (appointmentId, channel, offsetMinutes)
OptimizationRun	Proposal, schedule version hash, apply/undo state	index (clinicId, date)
ChatConversation, ChatMessage	Staff chat	index (conversationId, createdAt)
AuditLog	Who, what, when, before/after	append-only via trigger; index (clinicId, createdAt)

Database-level guarantees

- `appointments_no_overlap`: EXCLUDE USING gist (doctorId WITH =, tsrange(startsAt, endsAt) WITH &&) WHERE status NOT IN (CANCELLED, NO_SHOW).
- Audit rows cannot be deleted or have their content changed (trigger); ON DELETE SET NULL of actor/clinic is the only allowed update.
- Human-readable codes are backfilled for pre-existing rows in the migration, so it applies to populated databases.
- Performance indexes (migration 20261106180000_perf_indexes): appointments (clinicId, patientId, startsAt) and (clinicId, status, startsAt); queue_entries (clinicId, status, checkedInAt); chat_conversations (clinicId, updatedAt) and GIN on memberIds; audit_logs (clinicId, entity, createdAt).

5. Backend modules

Module	Responsibility	Highlights
auth, tenancy, users	Login, refresh, logout, reset, registration, invitations, workspace switching	Inherited and extended; rotating refresh with reuse detection
clinic	Reference data, rooms, specialties	Cached 60 s per clinic
patients	Directory, profile, duplicates, archive	Grouped lastVisit / nextAppointment; clinical notes hidden from receptionists (403)
doctors	CRUD, deactivate, hours, leave, slots, utilisation	Hours change → 409 HOURS_CONFLICT with list unless confirmed
appointments	CRUD, status, cancel, bulk, alternatives, locks, reminders retry	SERIALIZABLE + per-doctor mutex + exclusion constraint → 409 with ≤ 3 alternatives
queue, public	Check-in, call-next, start, complete, delay, priority, display, QR check-in	FOR UPDATE SKIP LOCKED; public routes expose numbers and rooms only
scheduling	Day grid, metrics, optimise, validate move, apply, undo	Version hash stale check; 15-minute undo; audit per move
analytics	Dashboard, reports, CSV, PDF	SQL aggregation, 60 s cache invalidated by events (0.13-0.22 s uncached, ~0.03 s cached locally); streamed CSV with BOM; dependency-free PDF writer
notifications	Feed, read state, reminder scheduling and processing	Idempotent on dedupeKey; SENT only after provider success
settings, audit	Clinic configuration, roles summary, audit browsing	No update/delete routes for audit
chat	Conversations and messages	Members only; plain-text bodies; cursor pagination
events, core, demo	Domain event bus, time-zone and slot core, settings reader, daily demo reset	Demo reset 404 unless DEMO_MODE and CRON_SECRET

Domain events

appointment.created / .rescheduled / .cancelled / .status_changed, queue.updated, schedule.optimized, patient.created, doctor.availability_changed, settings.updated. The four contract events are also published on Redis channel `riaya:events` (schema in packages/shared/src/schemas/realtime.ts).

6. API reference (summary)

All routes are under `/api`, documented in Swagger at `/api/docs` and in docs/API-CONTRACT.md. Errors: `{ code, message, details?, requestId }`; the message is localised from Accept-Language.

Area	Main routes
Auth	POST auth/login, auth/refresh, auth/logout, auth/forgot-password, auth/reset-password, auth/register, auth/switch-workspace; GET auth/me, auth/memberships
Reference	GET clinic/reference, rooms, specialties; POST rooms
Patients	GET/POST patients, GET patients/duplicates, GET/PATCH patients/:id, POST patients/:id/archive, GET patients/:id/appointments
Doctors	GET/POST doctors, GET/PATCH doctors/:id, POST :id/deactivate|activate, PUT :id/working-hours, POST/DELETE :id/leaves, GET :id/slots
Appointments	GET appointments, GET appointments/calendar, GET/PATCH appointments/:id, POST appointments, POST :id/status, POST :id/cancel, POST appointments/bulk, GET :id/alternatives, POST/DELETE appointments/locks, POST appointments/reminders/:id/retry
Queue	GET queue; POST queue/check-in, queue/call-next, queue/:id/start|complete|no-show|notify-delay|priority
Public	GET public/display/:clinicId; POST public/check-in/:clinicId (rate limited)
Scheduling	GET scheduling/day, scheduling/metrics; POST scheduling/optimize, scheduling/validate-move, scheduling/proposals/:id/apply|undo
Analytics	GET dashboard, analytics, analytics/export.csv, analytics/export.pdf
Notifications	GET notifications, notifications/unread-count, notifications/delivery-stats; POST notifications/:id/read, notifications/read-all; GET|POST notifications/reminders/process (cron)
Settings, audit	GET/PUT settings, GET settings/roles, GET audit
Chat	GET/POST chat/conversations, GET/POST chat/conversations/:id/messages, POST :id/read, GET chat/staff

Error codes

SLOT_CONFLICT, SLOT_LOCKED, OUTSIDE_WORKING_HOURS, ILLEGAL_TRANSITION, CANCEL_REASON_REQUIRED, CANCELLATION_POLICY, DUPLICATE_PATIENT, HOURS_CONFLICT, QUEUE_EMPTY, ALREADY_CHECKED_IN, STALE_PROPOSAL, UNDO_EXPIRED, CHECK_IN_NOT_FOUND, plus the auth and tenancy codes (UNAUTHENTICATED, FORBIDDEN, ACCOUNT_LOCKED, SECURITY_STATE_UNAVAILABLE, ...). Each has an English, Arabic and French message.

7. Core algorithms

Slot generation

For a doctor and a local date: WORK intervals of the weekday minus BREAK intervals minus leave, converted to UTC with the zone offset on that date (correct across daylight-saving changes and overnight shifts). Slots are cut on the clinic slot grid (default 15 min) from each available interval; a slot is unavailable when it overlaps a live appointment, a lock held by someone else, or starts in the past. Covered by 34 unit cases including midnight and DST days.

Conflict-safe booking

1) Optional Redis lock `slot:{doctor}:{start}` (5 min) while the user is in the wizard. 2) Per-doctor in-process mutex. 3) SERIALIZABLE transaction re-checks overlap of doctor and room. 4) The exclusion constraint rejects any overlap that slips through. Every rejection maps to 409 SLOT_CONFLICT with up to three alternatives: nearest free slots of the same doctor that day, then the earliest of other doctors with the same specialty. Verified with 50 parallel requests: exactly one 201.

Waiting-time model and optimiser

Per doctor, appointments in start order: service = duration × 1.15; $finish_i = max(start_i, finish_{i-1}) + service_i$; $wait_i = max(0, finish_{i-1} - start_i)$. Movable appointments are future SCHEDULED/CONFIRMED, not EMERGENCY and not locked. Candidates: same doctor or same specialty, on the slot grid, inside availability, within ±120 min, no overlap. Greedy local search, up to three passes, applying the single best strictly improving move, with deterministic tie-breaks. Objectives: minimise total estimated wait, minimise idle gaps, or minimise per-specialty utilisation variance. Property tests over 240 random days prove: no overlap, never outside hours, never moves protected appointments, deterministic, never worse. On the seeded benchmark day, minimise-wait reduces the estimated average wait from 6.9 to 1.4 minutes (8 moves).

Apply and undo

A proposal stores a SHA-256 of the day schedule. Apply re-computes it inside one transaction (mismatch → 409 STALE_PROPOSAL), re-validates every move, writes all moves and audit rows, or none. Undo is allowed for 15 minutes when the moved appointments are still where the run put them.

Queue ordering

Call-next order: priority (Emergency, Urgent, Normal), then earliest appointment time, then earliest check-in, selected with `FOR UPDATE SKIP LOCKED` so two receptionists never call the same patient. Waiting time is computed on the server from check-in time.

8. Front end

- **Shell:** navy sidebar (full, collapsed, drawer on mobile, state remembered), breadcrumbs, global search combobox, language switcher, notification bell, chat launcher, offline banner, error boundary, 404.
- **Data:** RTK Query endpoints injected per feature over the shared axios client (in-memory access token, one shared refresh request, Accept-Language). Mutations invalidate tags so lists, calendar and dashboard refresh without reload. Live screens poll (queue 5 s, notifications 20 s, dashboard 30 s) and can switch to Socket.IO when VITE_REALTIME_URL is set.
- **Design system:** tokens in index.css mirrored in tokens.ts (parity check), token lint forbidding colour literals and physical-direction utilities, primitives in src/common, feature building blocks in src/features/shared.
- **Internationalisation:** EN, AR (dir=rtl, mirrored icons, Latin digits for times and phone numbers), FR; per-feature catalogues mounted under the feature key; locale parity enforced in the build; Intl formatting in the clinic time zone.
- **Accessibility:** keyboard operation, visible focus, labelled controls, aria-live counters, status by icon and text (never colour alone), 40 px targets, a non-drag "Move to" alternative on the scheduling grid.
- **Performance:** route-level code splitting (calendar, scheduling, analytics, settings, public screens load on demand); polling paused when the tab is hidden.
- **Public screens:** /app/display/:clinicId (TV display: queue numbers and rooms only) and /app/check-in/:clinicId (mobile QR self check-in).

9. Security and privacy

Control	Implementation
Authentication	argon2 hashes; access JWT 15 min (algorithm pinned to HS256, exp validated; production refuses a missing or default secret); refresh 7 days in HttpOnly, Secure (prod), SameSite=Lax cookie; rotation with family revocation on reuse
Brute force and abuse	Five failures in 10 minutes lock the account for 15 minutes; generic error message. Per-route limits (429 RATE_LIMITED + Retry-After): login 10/min, refresh 30, forgot-password 5, reset 10, booking 20, slot locks 30, bulk 10, public check-in 10; trust proxy on Vercel so limits are per client
Authorisation	Capability matrix shared by API and UI; RolesGuard on every route; object-level checks (doctor own schedule, chat membership)
Tenant isolation	Clinic and organisation taken only from the verified token; every query filtered by clinicId; automated cross-tenant tests
Input	Global ValidationPipe (whitelist, forbid unknown), length and size limits, body limit 256 kB, parameterised SQL only
Output	Error filter returns code/message/requestId; Prisma P2025 → 404, P2002/P2003 → 409, anything else → generic 500; never stack traces or SQL; chat rendered as text
Headers	helmet with HSTS; CORS allow-list (no wildcard); SPA security headers on /app
Audit	Append-only audit rows for every create/update/delete and every status change, written in the same transaction
Privacy	Fictional data only; masked phones in lists; no names on the public display; logs redact authorization, cookies, passwords, phone, e-mail, date of birth, notes and address, and drop query strings
Secrets	Environment variables only (Vercel project env, sensitive); gitleaks in CI; .env files ignored

See docs/THREAT-MODEL.md for the STRIDE table and residual risks. Security tests cover rate limiting (login and booking return 429), forged alg:none tokens (401) and localised messages for every error code. Known residual risk in the hosted demo: without Redis, lockout counters are per serverless instance (SECURITY_STATE_MEMORY=true); refresh-token revocation remains enforced in PostgreSQL. Adding Upstash Redis (REDIS_URL) removes it.

10. Testing and quality

Suite	Scope	Result
server (Jest)	Unit: slots (34 cases incl. DST), state machine, utilisation, alternatives, queue order (12), metrics, optimiser (properties over 240 days), CSV, PDF, settings rules; e2e suites for auth, tenancy, isolation, audit	267 passed, line coverage 82%
backoffice (Vitest)	Wizard validation and draft, filters ↔ URL, calendar layout, search combobox, grid maths incl. RTL, insights state machine, analytics ranges, notification templates, settings validation, chat merge, queue rows, public display privacy, shell	184 passed
realtime (Jest)	Handshake auth, rooms, relay contract	46 passed
Concurrency	50 parallel bookings of one slot	1 × 201, 49 × 409 with alternatives
Journeys (browser)	Booking end to end; optimise → preview → apply; queue check-in → call → start → complete; RTL and 375 px	Verified locally and on the hosted demo
Static checks	ESLint (0 warnings), TypeScript strict, token lint, token parity, locale parity, Prettier, commitlint	Green

Definition of done

Lint, type-check and tests pass; UI checked at 375/768/1440 px in EN and AR; loading/empty/no-results/error states present; Swagger and contract updated; migration committed for schema changes; no secrets.

11. Deployment and operations

Item	Value
Web project	riaya → https://riaya-xi.vercel.app (root web/, framework Next.js)
API project	riaya-api → https://riaya-api.vercel.app (root server/, function api/index.js, 1 GB, 60 s)
Database	Neon PostgreSQL (Vercel Marketplace, us-east-1); pooled DATABASE_URL at runtime, direct URL for migrations
Build (web)	shared build → backoffice build with VITE_BASE=/app/ → copy to web/public/app → next build (API_ORIGIN at build time)
Build (API)	shared build → prisma generate (native + rhel-openssl-3.0.x) → nest build
Crons	06:00 UTC demo reset (daily re-seed); 06:30 UTC reminder processing
Key env (API)	DATABASE_URL, JWT_ACCESS_SECRET, CRON_SECRET, COOKIE_SECURE, CORS_ORIGINS, BACKOFFICE_URL, DEMO_MODE, SECURITY_STATE_MEMORY, optional REDIS_URL
Key env (web)	API_ORIGIN, NEXT_PUBLIC_BACKOFFICE_URL=/app

Release procedure

```
git push origin main
VERCEL_ORG_ID=<team> VERCEL_PROJECT_ID=<riaya-api>  vercel deploy --prod   # API first
VERCEL_ORG_ID=<team> VERCEL_PROJECT_ID=<riaya>      vercel deploy --prod   # then web
# schema change: DATABASE_URL=<direct> pnpm --filter server exec prisma migrate deploy (before the API deploy)
```

Runbook

- **Health:** GET /api/health returns database and cache state.
- **Logins return 503 SECURITY_STATE_UNAVAILABLE:** Redis unreachable and no SECURITY_STATE_MEMORY opt-in; restore Redis or set the flag consciously.
- **Stale proposals:** expected when the schedule changed; the UI offers Regenerate.
- **Re-seed demo now:** GET /api/demo/reset with Authorization: Bearer <CRON_SECRET>.
- **Logs:** Vercel runtime logs (structured pino JSON with request id).

12. Known limitations and roadmap

- Hosted demo has no Redis and no realtime service: per-instance lockout and caches, polling instead of push. Next: Upstash Redis + a container host for realtime/.
- Reminders use a database-backed scheduler and a mock provider instead of BullMQ (ADR 0002); a BullMQ worker can replace the processor without API changes.
- Maximise-utilisation objective only helps when several doctors share a specialty; the seeded Central Clinic has one doctor per specialty.
- Optional items from the Ri'aya document not built: billing and payments, digital intake forms, waitlist auto-fill, AI helpers, live floor heatmap.
- Load test (k6, 100 concurrent users) and Storybook stories for every component are not yet in the repository.

Demo accounts (fictional)

Role	E-mail	Password
Super admin	admin@riaya.local	RiayaAdmin123!
Clinic admin	clinic.admin@riaya.local	RiayaClinic123!
Doctor	doctor@riaya.local	RiayaDoctor123!
Receptionist	receptionist@riaya.local	RiayaReception123!